PRACTICAL SESSION 12

Task 1: Implement the Producer–Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
(base) saiabburi@Charishma-MacBook-Air ~ % mkdir -p ~/OS_Lab_Session12
(base) saiabburi@Charishma-MacBook-Air ~ % cd ~/OS_Lab_Session12
(base) saiabburi@Charishma-MacBook-Air OS_Lab_Session12 % nano producer_consumer.c
```

```
(base) saiabburi@Charishma-MacBook-Air OS_Lab_Session12 % ./producer_consumer
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Producer-Consumer completed successfully.
```

```
  UW PICO 5.09                                    File: producer_consumer.c

#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty_slots;
sem_t full_slots;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    (void)arg;

    for (int item = 1; item <= ITEMS; item++)
    {
        sem_wait(&empty_slots);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full_slots);
    }

    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&full_slots);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty_slots);
    }

    return NULL;
}

int main(void)
{
    pthread_t p, c;
```

```
    pthread_t p, c;

    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    pthread_mutex_destroy(&mutex);

    printf("Producer-Consumer completed successfully.\n");

    return 0;
}
```

```
(base) saiabburi@Charishma-MacBook-Air OS_Lab_Session12 % nano producer_consumer_mac.c
(base) saiabburi@Charishma-MacBook-Air OS_Lab_Session12 % clang -Wall -Wextra -pthread producer_consumer_mac.c -o producer_consumer_mac
(base) saiabburi@Charishma-MacBook-Air OS_Lab_Session12 % ./producer_consumer_mac
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Producer-Consumer completed successfully.
(base) saiabburi@Charishma-MacBook-Air OS_Lab_Session12 %
```

```
  UW PICO 5.09                                File: producer_consumer_mac.c


#include <stdio.h>
#include <pthread.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

int empty_slots = BUFFER_SIZE;
int full_slots = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t can_produce = PTHREAD_COND_INITIALIZER;
pthread_cond_t can_consume = PTHREAD_COND_INITIALIZER;

void *producer(void *arg)
{
    (void)arg;

    for (int item = 1; item <= ITEMS; item++)
    {
        pthread_mutex_lock(&mutex);

        while (empty_slots == 0)
            pthread_cond_wait(&can_produce, &mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        empty_slots--;
        full_slots++;

        printf("Produced: %d\n", item);

        pthread_cond_signal(&can_consume);
        pthread_mutex_unlock(&mutex);
    }

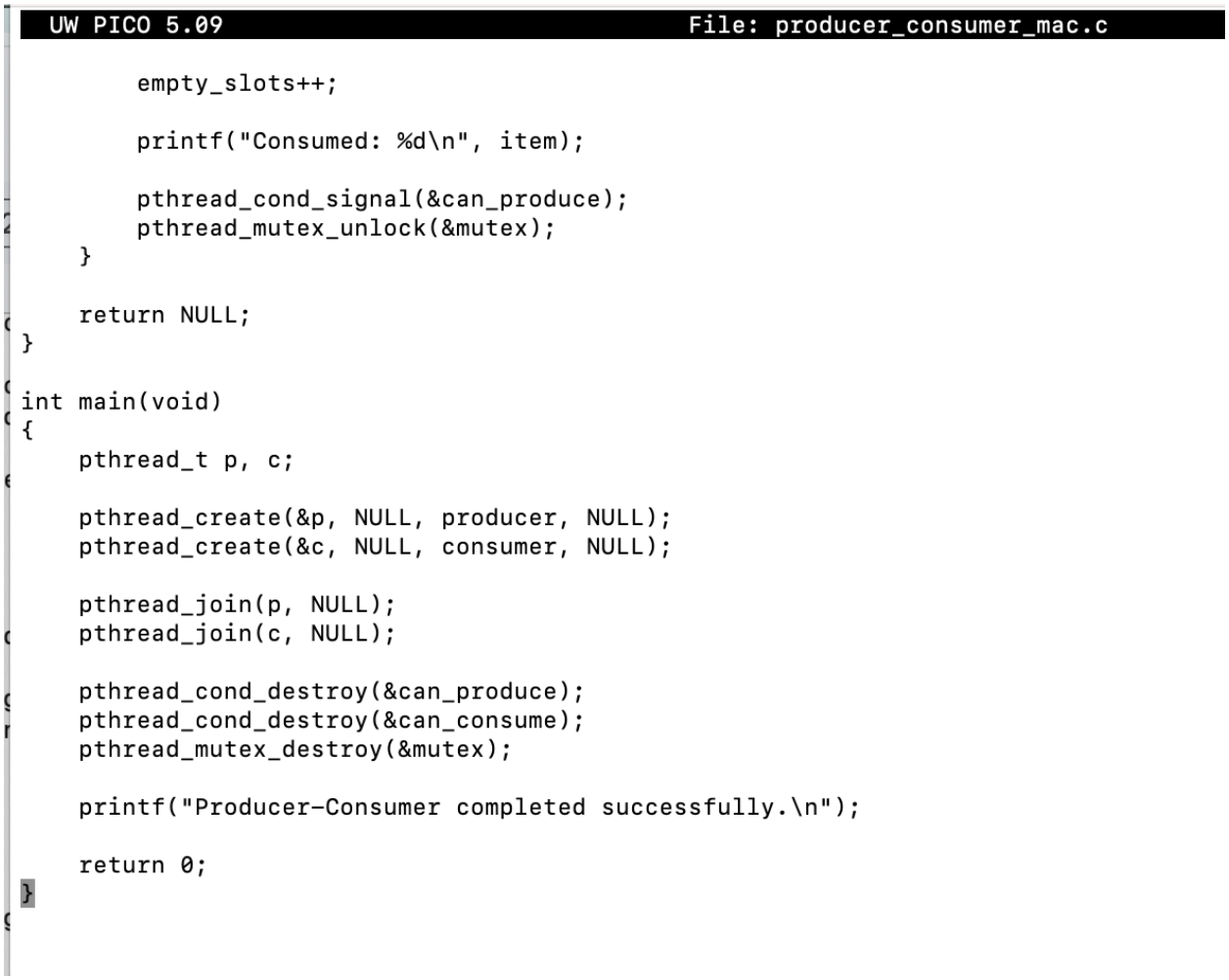
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITEMS; i++)
    {
        pthread_mutex_lock(&mutex);

        while (full_slots == 0)
            pthread_cond_wait(&can_consume, &mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        full_slots--;
```



```
  UW PICO 5.09                                File: producer_consumer_mac.c

        empty_slots++;

        printf("Consumed: %d\n", item);

        pthread_cond_signal(&can_produce);
        pthread_mutex_unlock(&mutex);
    }

    return NULL;
}

int main(void)
{
    pthread_t p, c;

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    pthread_cond_destroy(&can_produce);
    pthread_cond_destroy(&can_consume);
    pthread_mutex_destroy(&mutex);

    printf("Producer-Consumer completed successfully.\n");

    return 0;
}
```

Task 2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t sync_lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t sync_cond = PTHREAD_COND_INITIALIZER;

int ready = 0;

void *thread1(void *arg)
{
    (void)arg;

    pthread_mutex_lock(&A);
    printf("Thread 1 acquired Resource A\n");
    fflush(stdout);

    pthread_mutex_lock(&sync_lock);
    ready++;
    pthread_cond_broadcast(&sync_cond);
    while (ready < 2)
        pthread_cond_wait(&sync_cond, &sync_lock);
    pthread_mutex_unlock(&sync_lock);

    printf("Thread 1 waiting for Resource B\n");
    fflush(stdout);
    pthread_mutex_lock(&B);

    printf("Thread 1 acquired Resource B\n");

    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);

    return NULL;
}

void *thread2(void *arg)
{
    (void)arg;

    pthread_mutex_lock(&B);
    printf("Thread 2 acquired Resource B\n");
    fflush(stdout);

    pthread_mutex_lock(&sync_lock);
    ready++;
    pthread_cond_broadcast(&sync_cond);
    while (ready < 2)
        pthread_cond_wait(&sync_cond, &sync_lock);
    pthread_mutex_unlock(&sync_lock);

    printf("Thread 2 waiting for Resource A\n");
    fflush(stdout);
    pthread_mutex_lock(&A);
```

```
    printf("Thread 2 acquired Resource A\n");

    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);

    return NULL;
}

int main(void)
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

```
(base) saiabburi@Charishma-MacBook-Air ~ % mkdir 12
(base) saiabburi@Charishma-MacBook-Air ~ % cd 12
(base) saiabburi@Charishma-MacBook-Air 12 % nano deadlock_prevention.c
(base) saiabburi@Charishma-MacBook-Air 12 % clang -Wall -Wextra -pthread deadlock_prevention.c -o deadlock_prevention
(base) saiabburi@Charishma-MacBook-Air 12 % ./deadlock_prevention
Thread 1 acquired Resource A
Thread 1 acquired Resource B
Thread 1 completed its work
Thread 2 acquired Resource A
Thread 2 acquired Resource B
Thread 2 completed its work
Deadlock prevented using resource ordering.
(base) saiabburi@Charishma-MacBook-Air 12 %
```